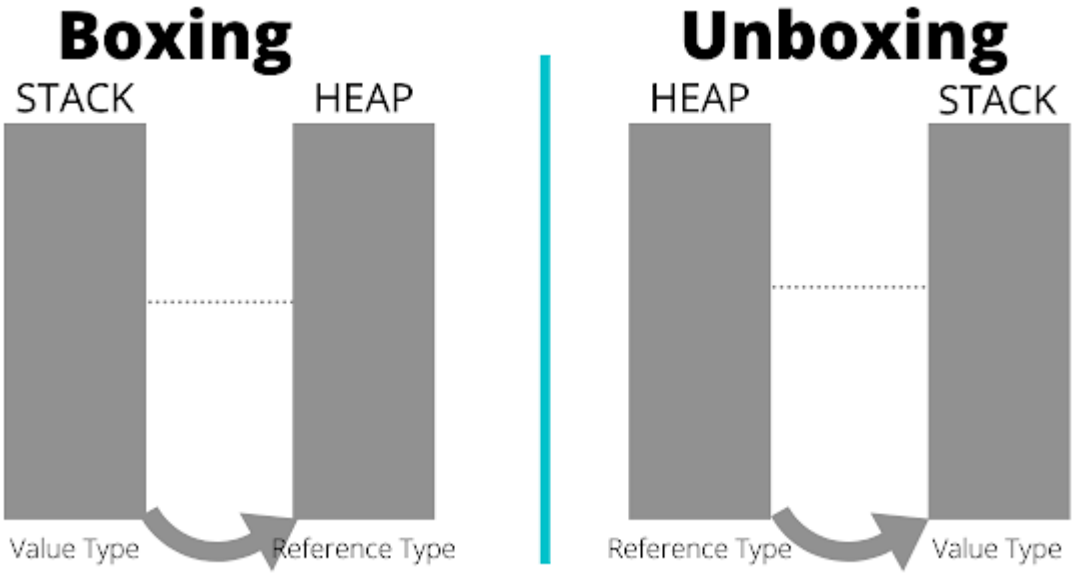
Boxing and Unboxing in C#

Boxing and unboxing are important concepts used in the C# type system. They are used to create a link between the two major data types in C#, the Value and Reference types. It simply refers to the allocation of a value type (For example, `int`, `char`, and so on.) on the heap rather than the stack. While working with the data types such as strings, objects, and arrays, we often have to convert the value types to the reference types or vice-versa. These conversion processes are generally referred to as boxing and unboxing.



Boxing is Implicit

Boxing is an implicit conversion of a value type (`int`, `char` and so on) to a reference type (object). In the boxing process, a value type is allocated on the heap rather than the stack.

Consider an example for boxing:

```
int x = 10;

object obj = i; //performs boxing
```

Here, the integer variable that is "`x`" is assigned to the object "`obj`". As a result, the object data type is a reference type and base class for all other classes in C#, eventually, an integer can be assigned to it. Thus, the process of converting from the integer to the object data type is called boxing.

Unboxing Must Be Explicit

Unboxing is the reverse of boxing. It is an explicit conversion of a reference type (which is being created by the boxing process); back to a value type. In the unboxing process, the boxed value type is unboxed from the heap and assigned to a value type that is being allocated on the stack.

Consider an example for unboxing:

```
object o = 10;
int i = (int)o; //performs unboxing
```

Here, a boxing conversion makes a copy of the value. So, changing the value of one variable will not impact others.

The casting of a boxed value is not permitted. For example:

```
int i = 10;
object o = i; // boxing
double d = (double)o; // runtime exception
```

Here, unboxing `int` value as double is not possible. If we want the result as double then, first we have to unbox the value as `int` (because the actual value was `int` when we boxed it) and then do casting, for example:

```
int i = 10;
object o = i; // boxing
double d = (double)(int)o; // valid
```

Note: Boxing and unboxing degrades the performance. So, avoid using it. Use generic classes with the generic interfaces to avoid boxing and unboxing. For example, use `List` instead of `ArrayList`.

Summary

Boxing and unboxing are the important concepts in the field of Object-Oriented Programming. However, both boxing and unboxing consume more time and memory, and they are computationally expensive. Therefore, it is always advised to avoid too much use of boxing and unboxing in the program and opt for generics. This helps to avoid performance problems in the boxing and unboxing operations.

******